

Assembly 8086

Il **linguaggio Assembly 8086** è un linguaggio di programmazione a basso livello progettato per il microprocessore **Intel 8086** e i suoi successori. Si basa su istruzioni mnemoniche che corrispondono direttamente alle operazioni eseguite dalla CPU, offrendo **pieno controllo sull'hardware** e massima efficienza in termini di prestazioni e utilizzo della memoria. A differenza dei linguaggi di alto livello, come C o Python, Assembly 8086 lavora direttamente con **registri, indirizzi di memoria e istruzioni macchina**, rendendolo essenziale per la **programmazione di sistema, driver hardware e ottimizzazione delle prestazioni**. Sebbene sia più complesso da scrivere e comprendere rispetto ai linguaggi moderni, è fondamentale per chi vuole approfondire il funzionamento dei microprocessori e l'architettura del computer.

Registri

Nel microprocessore **Intel 8086**, i registri si suddividono in diverse categorie, ognuna con una funzione specifica:

Registri di uso generale (General Purpose Registers)

Questi registri da **16 bit** possono essere suddivisi in due registri da **8 bit** (parte alta e parte bassa) e vengono utilizzati sia per operazioni generali sia per scopi specifici.

- **AX (Accumulator Register)** → Divisioni, moltiplicazioni e operazioni con I/O
 - **AH** (parte alta) e **AL** (parte bassa)
- **BX (Base Register)** → Indirizzamento di dati (può fungere da puntatore)
 - **BH** e **BL**
- **CX (Count Register)** → Contatore per cicli di loop e operazioni di shift/rotate
 - **CH** e **CL**
- **DX (Data Register)** → Operazioni di I/O, moltiplicazioni e divisioni
 - **DH** e **DL**

Registri di segmento (Segment Registers)

Questi registri determinano i segmenti di memoria in cui opera la CPU:

- **CS (Code Segment)** → Contiene l'indirizzo base del segmento di codice
- **DS (Data Segment)** → Contiene l'indirizzo base del segmento dati
- **SS (Stack Segment)** → Contiene l'indirizzo base dello stack
- **ES (Extra Segment)** → Usato per operazioni di memoria aggiuntive (es. operazioni su stringhe)

Registri di puntatori e indice (Pointer and Index Registers)

Questi registri servono per gestire lo stack e per l'indirizzamento:

- **SP (Stack Pointer)** → Punta alla cima dello stack (relativo a SS)
- **BP (Base Pointer)** → Usato per accedere ai dati nello stack
- **SI (Source Index)** → Usato per operazioni con stringhe e indirizzamento basato su DS
- **DI (Destination Index)** → Usato per operazioni con stringhe e indirizzamento basato su ES

Registro del puntatore di istruzioni

- **IP (Instruction Pointer)** → Contiene l'indirizzo dell'**istruzione successiva** da eseguire nel segmento codice (CS).

Registro di flag (Flag Register)

È un registro di **16 bit** che contiene vari bit di stato e controllo:

- **ZF (Zero Flag)** → Impostato se il risultato è zero
- **SF (Sign Flag)** → Indica il segno del risultato
- **CF (Carry Flag)** → Impostato in caso di overflow nei calcoli
- **OF (Overflow Flag)** → Indica overflow in operazioni con segno

- **PF (Parity Flag)** → Indica se il numero di bit a 1 è pari
- **AF (Auxiliary Carry Flag)** → Usato per operazioni BCD
- **TF (Trap Flag)** → Usato per il debug passo-passo
- **IF (Interrupt Flag)** → Abilita o disabilita gli interrupt
- **DF (Direction Flag)** → Determina la direzione delle operazioni sulle stringhe

Segmenti

L'**8086** utilizza un'**architettura segmentata**, il che significa che la memoria è suddivisa in blocchi di **64 KB** chiamati **segmenti**. Ogni segmento viene referenziato da un registro **segmento** e un registro **offset**.

I principali segmenti utilizzati in un programma **8086** sono:

1. **Segmento Codice (Code Segment - CS)**
2. **Segmento Dati (Data Segment - DS)**
3. **Segmento Stack (Stack Segment - SS)**
4. **Segmento Extra (Extra Segment - ES)**

Segmento Codice (CS - Code Segment)

- Contiene il **codice eseguibile** del programma.
- Il **registro CS (Code Segment)** punta all'inizio di questo segmento.
- L'indirizzo effettivo dell'istruzione è dato da:

$$\text{Indirizzo Effettivo} = (\text{CS} \times 16) + \text{IP}$$
 dove **IP (Instruction Pointer)** contiene l'offset dell'istruzione corrente.

Esempio di utilizzo:

`MOV AX, CS` ; Carica il valore del segmento codice in AX

In genere, **CS** viene impostato dal sistema e non viene modificato a meno di un cambio di

Segmento Dati (DS - Data Segment)

- Contiene **variabili, array, stringhe** e altri dati del programma.
- Il **registro DS (Data Segment)** punta all'inizio del segmento dati.
- Gli indirizzi effettivi dei dati si calcolano così:
$$\text{Indirizzo Effettivo} = (\text{DS} \times 16) + \text{Offset}$$

Indirizzo Effettivo = (DS \times 16) + Offset
dove **Offset** è contenuto in registri come **SI, DI o BX**.

Esempio di utilizzo:

```
MOV AX, DS ; Carica il valore del segmento dati in AX
MOV SI, 1000H ; Offset all'interno del segmento dati
MOV AL, [SI] ; Legge il byte all'indirizzo DS:1000H
```

Se si vogliono usare più segmenti dati, si deve **modificare DS** con istruzioni come MOV o LDS.

Segmento Stack (SS - Stack Segment)

- Contiene **lo stack del programma**, utilizzato per chiamate di subroutine, salvataggio di dati temporanei e ritorni da funzioni.
- Il **registro SS (Stack Segment)** indica l'inizio dello stack.
- Il **registro SP (Stack Pointer)** tiene traccia della posizione corrente dello stack.

Esempio di utilizzo:

```
MOV AX, SS ; Carica il valore del segmento stack in AX
MOV SP, 0FFFFH ; Imposta la cima dello stack
PUSH AX ; Salva AX nello stack (SP viene decrementato)
POP BX ; Recupera AX dallo stack in BX (SP viene incrementato)
```

Lo stack cresce verso **indirizzi inferiori** in memoria.

Segmento Extra (ES - Extra Segment)

- Usato per **operazioni su stringhe e trasferimenti di dati**.
- Il **registro ES (Extra Segment)** punta all'inizio di questo segmento.
- Usato insieme a **DI (Destination Index)** per operazioni con stringhe.

Esempio di utilizzo (movimenti tra segmenti):

```
MOV ES, AX ; Imposta il segmento extra
MOV DI, 200H ; Offset di destinazione
MOV SI, 100H ; Offset sorgente in DS
MOV CX, 10 ; Numero di byte da copiare
REP MOVSB ; Copia CX byte da DS:SI a ES:DI
```

Il prefisso REP ripete MOVSB (movimento di byte) fino a che **CX diventa zero**.

Come Usare i Segmenti

Caso d'uso	Segmento	Registro Segmento	Registro Offset
Esecuzione del codice	Code Segment	CS	IP
Lettura/Scrittura dati	Data Segment	DS	SI, DI, BX
Uso dello stack	Stack Segment	SS	SP, BP
Operazioni su stringhe	Extra Segment	ES	DI

L'architettura segmentata dell'8086 consente di accedere fino a **1 MB di memoria** nonostante abbia registri a **16 bit**. Per un uso efficace della memoria: **Usa CS + IP per il codice**

- **Usa DS + SI/DI/BX per i dati**
- **Usa SS + SP per lo stack**
- **Usa ES per operazioni speciali (es. stringhe)**

Modello di programma 8086

Modello	Segmenti Codice & Dati	Dimensione massima dati	Uso comune
---------	------------------------	-------------------------	------------

TINY	Unico segmento per tutto	64 KB	.COM files, TSR
-------------	--------------------------	-------	-----------------

SMALL	Un solo segmento per codice e uno per dati	64 KB dati	Programmi .EXE piccoli
--------------	--	------------	------------------------

MEDIUM	Codice multiplo, dati unico	64 KB dati	Programmi .EXE medi
---------------	-----------------------------	------------	---------------------

COMPACT	Unico codice, dati multipli	> 64 KB dati	Database, file manager
----------------	-----------------------------	--------------	------------------------

LARGE	Codice e dati multipli	> 64 KB dati	Programmi grandi
--------------	------------------------	--------------	------------------

HUGE	Simile a LARGE ma con gestione speciale delle strutture	> 64 KB	Raramente usato
-------------	---	---------	-----------------

Stack

Lo Stack nell'8086

Lo **stack** è un'area della memoria **LIFO** (Last In, First Out) che viene utilizzata per memorizzare dati temporanei, come variabili locali, indirizzi di ritorno dalle subroutine e informazioni di stato del programma. È fondamentale per la gestione delle chiamate di funzione e degli interrupt.

Lo stack è fondamentale nell'8086 per:

- **Salvare e recuperare dati temporanei**
- **Gestire subroutine e funzioni**
- **Memorizzare il contesto del programma (FLAGS, IP, ecc.)**

Struttura dello Stack

- L'**8086** utilizza il registro **SS (Stack Segment)** per indicare il segmento di memoria dove si trova lo stack.
- Il registro **SP (Stack Pointer)** tiene traccia della posizione attuale della cima dello stack.
- Lo stack cresce **verso indirizzi di memoria inferiori** (decremento di SP).
- Gli elementi nello stack vengono memorizzati in **parole da 16 bit** (due byte alla volta).

Operazioni sullo Stack

L'8086 permette diverse operazioni sullo stack, tra cui:

➤ **PUSH (Inserimento nello stack)**

- Salva un valore nello stack e decrementa lo Stack Pointer (SP).

Esempio:

PUSH AX ; Salva il valore di AX nello stack

- **Effetto:**
 - $SP = SP - 2$
 - Il valore di AX viene memorizzato all'indirizzo SS:SP.

➤ POP (Estrazione dallo stack)

- Recupera un valore dallo stack e incrementa SP.

Esempio:

POP BX ; Recupera un valore dallo stack e lo mette in BX

- **Effetto:**
 - Il valore all'indirizzo SS:SP viene caricato in BX.
 - $SP = SP + 2$

➤ CALL e RET (Chiamata e ritorno da una subroutine)

CALL salva l'indirizzo di ritorno nello stack e salta alla subroutine.

CALL subroutine

- **Effetto:**
 - L'indirizzo di ritorno (IP) viene salvato nello stack.
 - Il programma salta all'indirizzo della subroutine.

RET recupera l'indirizzo di ritorno dallo stack e lo carica in IP.

RET

- **Effetto:**
 - L'indirizzo viene prelevato dallo stack e caricato in IP.

- L'esecuzione riprende dal punto in cui la subroutine è stata chiamata.

➤ PUSHF e POPF (Salvataggio e ripristino dei flag)

PUSHF salva il registro **FLAGS** nello stack.

POPF ripristina i flag dallo stack.

Esempio di utilizzo dello Stack

Ecco un esempio pratico:

```
MOV AX, 1234H ; Carica un valore in AX
PUSH AX       ; Salva AX nello stack
MOV BX, 5678H ; Carica un valore in BX
PUSH BX       ; Salva BX nello stack

POP CX        ; Recupera BX in CX
POP DX        ; Recupera AX in DX
```

Alla fine:

- **CX** = 5678H (l'ultimo valore salvato è il primo recuperato)
- **DX** = 1234H (il primo valore salvato è l'ultimo recuperato)

Macro e Funzioni

In Assembly, sia le **macro** che le **funzioni** sono utilizzate per strutturare il codice in modo riutilizzabile e modulare. Tuttavia, ci sono differenze fondamentali nel loro funzionamento e nell'uso dei parametri, della gestione dello stack e nel salvataggio dei registri. Ecco un confronto panoramico tra le due:

Definizione e Scopo

- **Macro:**
 - Una **macro** è una sequenza di istruzioni che viene espansa dal preprocessore al momento della compilazione.
 - Le macro **non sono funzioni reali**; semplicemente, il codice definito nella macro viene inserito al posto della chiamata ogni volta che viene invocata.
 - Le macro non richiedono un **salvataggio esplicito dei registri** o l'uso dello **stack**, poiché non c'è un "passaggio" di controllo come nelle funzioni.
- **Funzioni:**
 - Una **funzione** in Assembly è una sezione di codice eseguibile che può essere invocata durante l'esecuzione del programma.
 - Le funzioni operano tramite **chiamate** esplicite e possono avere **parametri di ingresso e di ritorno**.
 - Le funzioni utilizzano **stack** e **salvataggio dei registri** per mantenere la coerenza dello stato del programma.

Parametri di Ingresso e Ritorno:

- **Macro:**
 - Le macro **non accettano parametri** nel senso tradizionale. I parametri vengono passati come **argomenti** alla macro, ma questi vengono trattati come semplici **sostituzioni** al momento dell'espansione del codice.
 - Ad esempio, se definisci una macro che accetta un valore come parametro, quando invoci la macro, quel parametro sarà sostituito direttamente nel codice.
- **Funzioni:**

- Le funzioni **usano parametri di ingresso** che vengono generalmente passati attraverso registri (ad esempio ax, bx, dx) o attraverso lo **stack**.
- I **parametri di ritorno** possono essere passati tramite registri, ma nel caso di valori complessi o più di un valore, possono essere restituiti tramite lo **stack**.

Stack e Salvataggio dei Registri:

- **Macro:**

- Le macro **non utilizzano lo stack**. Non è necessario preoccuparsi del salvataggio o ripristino dei registri perché, essendo solo un'espansione del codice, non ci sono chiamate o ritorni come nelle funzioni.
- In pratica, una macro viene espansa direttamente nel punto in cui viene chiamata, ed è come se il codice fosse scritto manualmente.

- **Funzioni:**

- Le funzioni, quando vengono chiamate, **utilizzano lo stack** per **salvare lo stato** dei registri, in modo che il programma possa riprendere correttamente l'esecuzione dopo che la funzione ha terminato.
- All'inizio di una funzione, generalmente, si salva lo stato dei registri che potrebbero essere utilizzati all'interno della funzione e si riservano spazi nello stack per i parametri di ingresso (se non passati tramite registri).
- Dopo l'esecuzione della funzione, i registri devono essere **ripristinati** e lo stack deve essere **ripulito** prima di tornare al chiamante.

Ecco un **esempio completo** di una **macro** e di una **funzione (procedura)** in Assembly **8086**, entrambe fanno la **somma di due numeri** e restituiscono il risultato nel registro **AX**.

Macro per la Somma di Due Numeri

```
SUM_MACRO MACRO num1, num2
    MOV AX, num1 ; Carica il primo numero in AX
    ADD AX, num2 ; Aggiunge il secondo numero
ENDM
```

```
SECTION .TEXT
START:
    SUM_MACRO 5, 10 ; AX = 5 + 10
    ; Qui AX contiene 15
```

◆ Spiegazione della Macro

- SUM_MACRO num1, num2 accetta due numeri come parametri.
- MOV AX, num1 carica il primo numero in AX.
- ADD AX, num2 somma il secondo numero.

Svantaggio: ogni volta che la macro viene usata, il codice viene copiato direttamente, aumentando la dimensione del programma.

Funzione (Procedura) per la Somma di Due Numeri

```
SECTION .DATA
```

```
NUM1 DW 5
```

```
NUM2 DW 10
```

```
SECTION .TEXT
```

```
SUM_FUNCTION PROC
```

```
    PUSH BP          ; Salva il valore di BP
```

```
    MOV BP, SP       ; Imposta BP per accedere ai parametri
```

```
    MOV AX, [BP+4]    ; Carica il primo numero in AX (parametro 1)
```

```
    ADD AX, [BP+6]    ; Aggiunge il secondo numero (parametro 2)
```

```
    POP BP           ; Ripristina BP
```

```
    RET 4            ; Restituisce il valore e rimuove i parametri
```

```
dallo stack
```

```
SUM_FUNCTION ENDP
```

```
START:
```

```
    PUSH NUM1
```

```
    PUSH NUM2
```

```
    CALL SUM_FUNCTION ; Chiama la funzione con due parametri
```

```
    ; Qui AX contiene 15
```

♦ Spiegazione della Funzione

- PUSH NUM1 e PUSH NUM2 mettono i numeri nello **stack**.
- CALL SUM_FUNCTION chiama la funzione.
- MOV AX, [BP+4] e MOV [BP+6] recuperano i numeri dallo stack.

- RET 4 rimuove i parametri dallo stack e ritorna.

Vantaggio: il codice della funzione è memorizzato **una sola volta**, risparmiando memoria.

Differenze tra Macro e Funzione

Caratteristica	Macro	Funzione
Codice generato	Copia tutto il codice ogni volta	Usa un'unica copia del codice
Prestazioni	Più veloce (nessun overhead di chiamata)	Più lenta (CALL e RET)
Uso della memoria	Più grande	Più efficiente
Passaggio di parametri	Valori diretti	Stack

Vantaggi e Svantaggi:

- **Macro:**
 - **Vantaggi:**
 - **Velocità:** Poiché il codice della macro viene "inserito" direttamente nel punto in cui viene chiamato, non c'è un overhead dovuto alla gestione dello stack o delle chiamate di funzione.
 - **Flessibilità:** Le macro possono generare codice personalizzato con parametri differenti.
 - **Svantaggi:**
 - **Espansione del codice:** Le macro possono portare a una crescita del codice se vengono chiamate ripetutamente, aumentando così la dimensione del programma.
 - **Debugging difficile:** Poiché il codice viene espanso, il processo di debugging può diventare difficile.
- **Funzioni:**
 - **Vantaggi:**
 - **Organizzazione:** Le funzioni offrono una struttura migliore e consentono una gestione più facile del codice.
 - **Riutilizzo:** Le funzioni possono essere chiamate da diverse parti del programma senza duplicare il codice.

- **Gestione automatica dello stack:** Le funzioni gestiscono automaticamente lo stack e i registri, evitando errori di accesso alla memoria.
- **Svantaggi:**
 - **Overhead:** Le funzioni richiedono una chiamata e un ritorno, che comportano un certo overhead (ad esempio, la gestione dello stack e dei registri).
 - **Minor flessibilità** rispetto alle macro, poiché non si può modificare direttamente il codice generato da una funzione senza modificare il corpo della funzione stessa.

Quando Usare Macro o le Funzioni:

- **Macro:**
 - Quando hai bisogno di un codice che deve essere **inserito ripetutamente** in diverse parti del programma.
 - Quando vuoi che il codice sia **più veloce** senza preoccuparsi del risparmio di memoria, poiché ogni invocazione di macro copia letteralmente il codice.
 - **Funzioni:**
 - Quando hai bisogno di **gestire parametri** o **effettuare calcoli complessi** che richiedono la memorizzazione dei risultati.
 - Quando il **riutilizzo del codice** è necessario, ma vuoi evitare l'espansione del codice e preferisci una gestione centralizzata della logica.
-
- Se il tuo programma è piccolo e deve essere **super veloce**, usa una **macro**.
 - Se il tuo programma è **lungo e ripetitivo**, usa una **funzione** per **ottimizzare la memoria**.

In sintesi, la principale differenza tra **macro** e **funzioni** in Assembly risiede nella **gestione del flusso di controllo** e nella **memorizzazione dei dati**. Le macro sono più veloci e flessibili, ma possono portare a un codice più grande e complesso da gestire. Le funzioni, d'altra parte, sono più strutturate e riutilizzabili, ma con un piccolo overhead in termini di performance e complessità di gestione dello stack e dei registri.

Ordine di Grandezza

L'**ordine di grandezza** di un algoritmo, spesso indicato con il termine **complessità computazionale**, rappresenta una misura della **quantità di risorse** (tempo o spazio) che un algoritmo richiede al crescere delle dimensioni dell'input. In altre parole, l'ordine di grandezza descrive come il tempo di esecuzione (o la memoria utilizzata) di un algoritmo cambia in relazione alla dimensione dell'input, di solito denotato come **n**.

Cos'è l'Ordine di Grandezza (Complessità Temporale)?

L'ordine di grandezza è una **notazione asintotica** che permette di classificare gli algoritmi in base al loro comportamento al crescere delle dimensioni dell'input. Le notazioni più comuni usate per esprimere la complessità di un algoritmo sono:

- **O (Big O)**: Descrive il limite superiore del tempo di esecuzione, ovvero il massimo tempo che un algoritmo può impiegare in funzione della dimensione dell'input.
- **Ω (Big Omega)**: Descrive il limite inferiore, cioè il miglior tempo di esecuzione possibile.
- **Θ (Big Theta)**: Descrive il caso medio, cioè il tempo di esecuzione in cui sia il limite superiore che quello inferiore sono uguali.
- **o (little o) e ω (little omega)**: Descrivono limiti più precisi, ma sono meno usati rispetto alle altre notazioni.

Come si Calcola l'Ordine di Grandezza (Complessità Asintotica)?

Per calcolare l'ordine di grandezza di un algoritmo, si segue generalmente un approccio in tre fasi:

a) Identificare il Passo Dominante:

Quando analizziamo un algoritmo, dobbiamo determinare quale operazione (o passo) è quella che **domina il tempo di esecuzione** per l'input di dimensione **n**. Questo viene fatto analizzando il numero di iterazioni nei cicli, la profondità delle ricorsioni, e così via.

b) Trascurare i Termini Minori:

Nel calcolo della complessità, si trascurano i termini che crescono più lentamente in confronto a quello che domina. Ad esempio, se un algoritmo ha una complessità di $O(n^2 + 3n + 5)$, si considera solo $O(n^2)$, poiché n^2 è il termine dominante.

c) Utilizzare la Notazione Asintotica:

Una volta identificato il passo dominante, si utilizza la notazione di complessità asintotica per descrivere la crescita dell'algoritmo in termini di n .

Esempi di Complessità Asintotiche Comuni:

$O(1)$: Complessità costante. Il tempo di esecuzione non dipende dalla dimensione dell'input. Ad esempio, accedere a un elemento di un array tramite un indice.

```
mov al, [array + index] ;  $O(1)$ 
```

$O(\log n)$: Complessità logaritmica. Il tempo di esecuzione cresce logaritmicamente al crescere dell'input. Un esempio classico è la **ricerca binaria**.

```
; Ricerca binaria ( $\log n$ )
```

$O(n)$: Complessità lineare. Il tempo di esecuzione cresce in modo proporzionale alla dimensione dell'input. Ad esempio, un ciclo che scorre tutti gli elementi di un array.

```
; Ciclo che percorre tutti gli elementi di un array  
for i = 0 to n  
    do something
```

$O(n \log n)$: Complessità logaritmica-lineare. Esempi includono algoritmi di ordinamento efficienti come **Merge Sort** e **Quick Sort**.

```
; Algoritmi di ordinamento come Merge Sort ( $n \log n$ )
```

$O(n^2)$: Complessità quadratica. Il tempo di esecuzione cresce in modo proporzionale al quadrato della dimensione dell'input. Ad esempio, il **Bubble Sort**.

```
; Algoritmo di ordinamento come Bubble Sort ( $n^2$ )
```

$O(2^n)$: Complessità esponenziale. Il tempo di esecuzione cresce esponenzialmente con la dimensione dell'input. Un esempio è la soluzione **naive** di problemi di ricerca o combinatoria, come il calcolo di tutte le possibili combinazioni.

```
; Algoritmo che esplora tutte le combinazioni ( $2^n$ )
```


$O(n!)$: Complessità fattoriale. Il tempo di esecuzione cresce in modo fattoriale con la dimensione dell'input. Un esempio è il problema del **viaggio del commesso viaggiatore (TSP)**.

; Soluzione del problema del viaggio del commesso viaggiatore (TSP) con ricerca esaustiva

Analisi di un Algoritmo: Un Esempio Concreto

Consideriamo il seguente esempio di algoritmo di ordinamento **Bubble Sort**:

```
; Bubble Sort -  $O(n^2)$ 
for i = 0 to n - 1
  for j = 0 to n - i - 1
    if arr[j] > arr[j + 1]
      swap(arr[j], arr[j + 1])
```

a) Identificazione del passo dominante:

- Il ciclo esterno (su i) ha n iterazioni.
- Il ciclo interno (su j) ha al massimo $n-i-1$ iterazioni.
- Il numero totale di confronti è quindi circa $n \times (n-1)/2 \times (n-1)/2$, che è **$O(n^2)$** .

b) Ignorare i termini minori:

La complessità del ciclo interno è ignorata e si considera solo $O(n^2)$, che è il termine dominante.

Riassunto dei Passaggi per Calcolare l'Ordine di Grandezza:

1. **Analizzare la struttura dell'algoritmo** e determinare i cicli, le ricorsioni e le operazioni di base.
2. **Identificare il passo dominante** che determina il tempo di esecuzione asintotico.
3. **Utilizzare la notazione asintotica** per rappresentare la complessità (ad esempio $O(n^2)$, $O(n \log n)$, etc.).
4. **Ignorare i termini minori** e le costanti.

L'ordine di grandezza di un algoritmo è una misura essenziale per valutare la sua efficienza in relazione alla crescita dell'input. Utilizzando la notazione asintotica come $O(n)$, $O(n^2)$, o $O(\log n)$, possiamo prevedere come un algoritmo si comporterà con input sempre più grandi, permettendo così di scegliere l'algoritmo più adatto alle specifiche esigenze di prestazione e risorse.

